DESIGN DOCUMENT

List of Contributors

Hatice Melike Aydin 231101063

İclal Zeynep Kaya 231101073

Anıl Özişler 211101081

Task Matrix

Hatice Melike Aydin	Anıl Özişler	İclal Zeynep Kaya
Use Case Support in Design	System Overview	Implementation Details
Design Decisions	Design Decisions	Design Decisions

Table Of Contents

1) System Overview

➢ Project Description

Synkly is a web application designed to help university students find common free time slots with their friends based on their weekly schedule. Students studying on the same campus often have overlapping class schedules and personal timetables. Although they would like to socialize and have meals together, finding common free time slots can be challenging. This project offers a simple web solution that automatically detects shared available times from students' schedules and suggests suitable meeting slots.

➢ System Architecture

➢ **Technology Stack**

Frontend: HTML, CSS, React, JavaScript

Backend & Database: Firebase

Version Control: GitHub

Communication: Slack

2) Implementation Details

➢ **Codebase Structure**

```
Synkly/
├── ProjectOutputPackagePA1
│   ├── ProjectDefinitionDocument.pdf
│   ├── ProjectPlanDocument.pdf
│   ├── ProjectRequirementsDocument.pdf
│
├── ProjectOutputPackagePA2
│   ├── DesignDocument.pdf
│   ├── QualityAssurancePlan.pdf
│
├── website
│   ├── profile.jpg
│   ├── background.jpg
│   ├── firebase.js
│   ├── index.html
│   ├── login.html
│   ├── login.js
│   ├── register.js
│   ├── script.js
│   ├── profile.html
│   ├── scriptProfile.js
│   ├── signup.html
│   ├── style.css
│   ├── style2.css
│   ├── style3.css
│   ├── styleProfile.css
```

➢ **Key Implementations**

1. **Schedule Management Module**

    a.Users upload, edit, and delete their weekly schedules through the React interface.

    b. All schedule data is stored securely in Firebase Firestore.

2. **User Authentication**

    a. Firebase Authentication handles secure user sign-up, login, and session management.

    b. Secure authentication with email and password

    c. Password validation includes minimum length, uppercase letters, and numeric requirements.

3. **Free Time Matching Algorithm**

    a. Automatically calculates users' free time blocks based on their weekly schedules.

    b. Matches the user's availability with selected friends to find overlapping free hours.

4. **Dining Recommendation Module**
    a. Suggests on-campus dining locations suitable for the matched free time window.

5. **Invitation & Interaction System**
    a. Users can send meeting or dining invitations to friends directly through the system.
    b. Invitations include selected time slots and recommended dining options.
    c. The system tracks accepted, pending, and declined invitations.

6. **User Profile Management**
    a. Profile data is stored securely in Firebase.

➢ **Component Interfaces**

1. **Authentication Components**

- **Login Component**
  - **Input Fields:** Username and password input fields.
  - **Button:** Submit button for logging in.
  - **Link:** Redirects to the signup page for account creation.
  - **Functionality:** Handles user authentication and redirects to the main page upon successful login**.**
- **Signup Component**
  - **Input Fields:** Usernamel, password, and password confirmation fields.
  - **Buttons**: Signup and cancel buttons.
  - **Functionality:** Creates a new user account and redirects to the login page upon successful registration

2. **Main Application Components**

- **Schedule Input Component**
  - **Input Field:** Time-slot selectors for entering weekly course hours (day, start time, end time)
  - **Buttons:** "Add Slot" button to add a course entry.

    "Save Schedule" button to store the schedule in FirebaseFirestore
  - **Functionality:** Allows users to input, edit, and save their weekly schedules. Ensures no overlapping time slots and updates the user's availability data.

- **Match Results Component**
  - **List Items:** Displays a list of matched friends with overlapping free time slots.
  - **Functionality:** Lists the users with whom free time matches exist using data from Firestore and the matching algorithm.

- **Match Details Modal**
  - **Content:** Displays shared free time slots, dining venues.
  - **Button:** Close button to dismiss the modal.

  - **Functionality:** Shows detailed information about the selected match and the ability to initiate an invitation.

3. Navigation Components

- **Logout Button**
  - **Functionality:** Logs out the user and redirects to the login page.

4. Styling Components

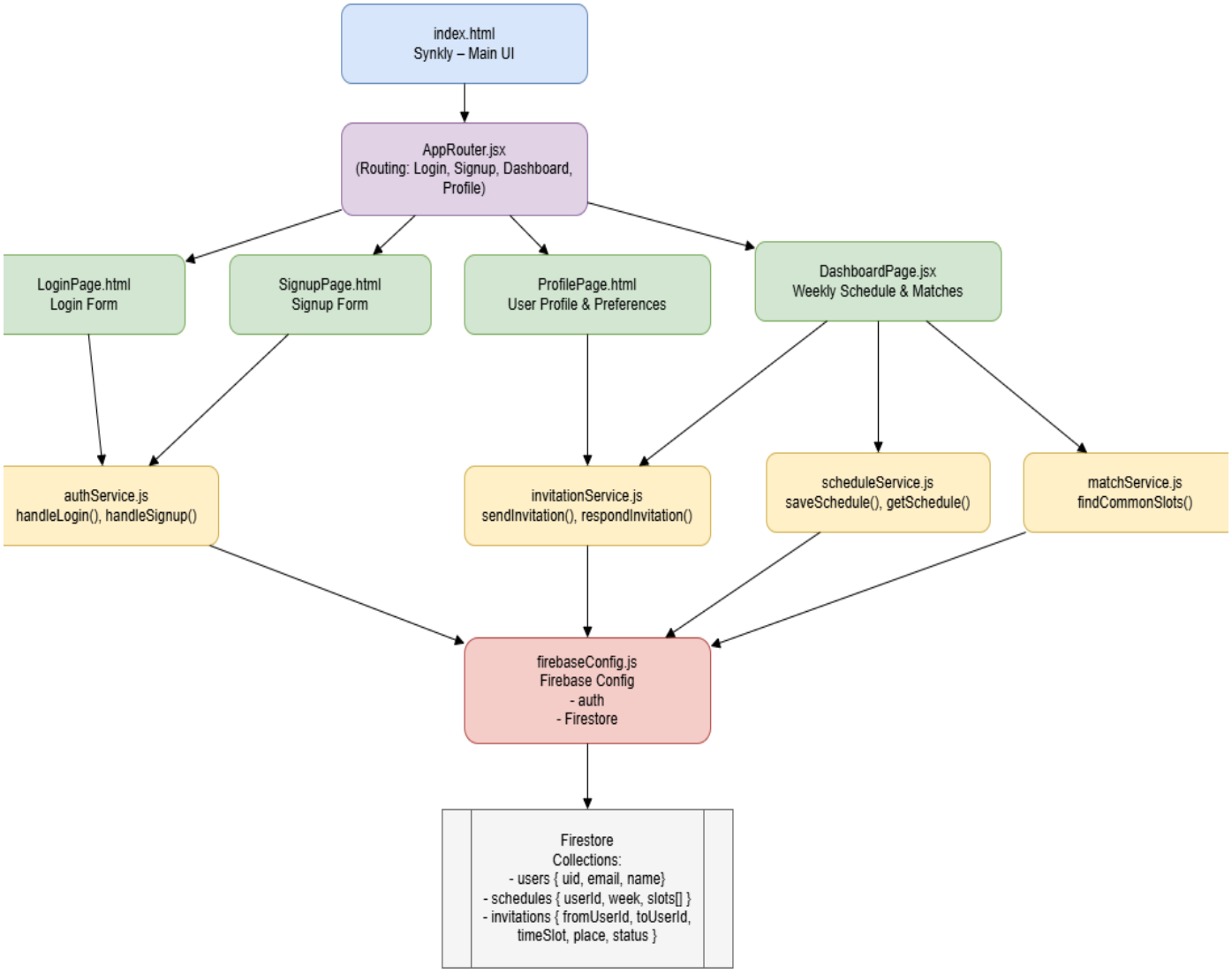
- **CSS Files**
  - **style.css:** Styles for the main page.
  - **style2.css:** Styles for the login page.
  - **style3.css:** Styles for the signup page.

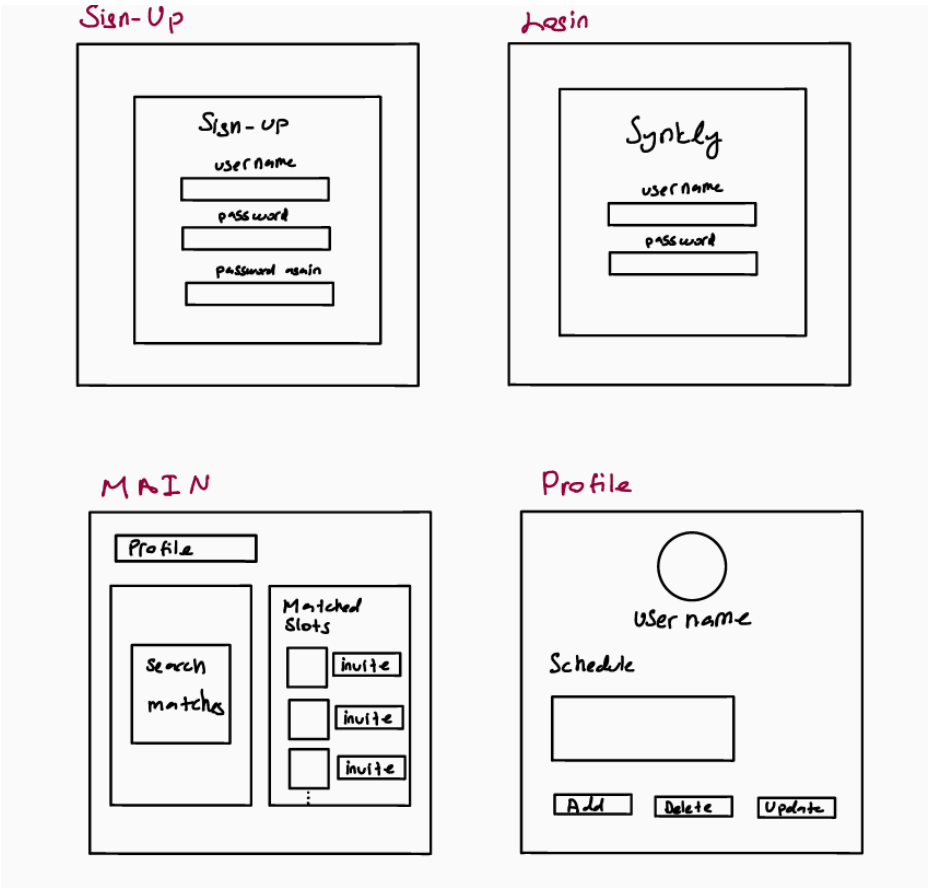
5. Firebase Integration

- **Authentication**

  - **Functionality:** Handles user registration and login using Firebase Authentication.

➢ Visual Interfaces





3) Use Case Support in Design

➢ Use Case Selection

1) Users must be able to create an account.
2) Users must be able to log in.
3) Users must be able to input and manage their weekly class schedules.
4) Users must be able to select available time slots and pre-defined dining places and the system must automatically detect overlapping free time slots among students.

➢ Requirement Mapping

Use Case	Requirement
User Account Creation	Users must create an account securely.
User Login	Users must be able to log in
Schedule Management	Users must be able to input their weekly class schedules and edit/delete schedules - the system must respect privacy through opt-in matching
Free-Time Matching	Users must select pre defined dining venues and the system must detect overlapping free time slots and display available students

➢ **Use Case Design**

- **User Account Creation**

Name	User Account Creation
Actors	Guest
Entry Condition	Person has internet connection and visits the website
Event Flow	1. Guest clicks on "Sign Up" 2. Inserts username, password (min 8 characters, uppercase, numbers) 3. Confirms password 4. Account is registered by the system 5. System directs the user to the login page
Exit Conditions	The guest now is registered and becomes a User.
Exception	User is already registered Username is already associated to another account Password doesn't meet requirements

- **User Login**

Name	User Login
Actors	User
Entry Condition	User is registered in the system
Event Flow	1. User connects to website 2. Enters username and password 3. Clicks on "Login" 4. The system checks credentials validity and directs the user to the main page if the information is valid

Exit Conditions	User successfully logged in and redirected to dashboard
Exception	Invalid credentials - error message displayed

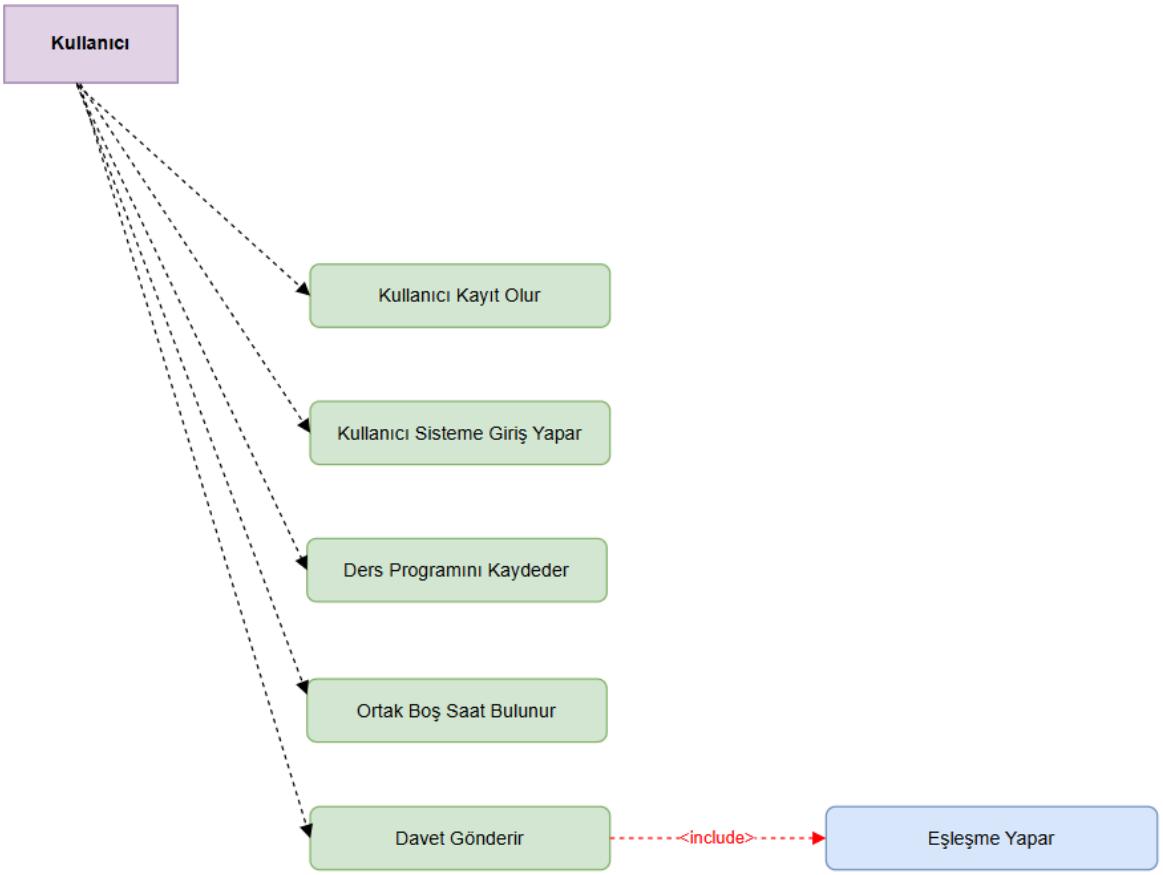
- **Schedule Management**

Name	Schedule Management
Actors	User
Entry Condition	User is authenticated and logged in
Event Flow	1. User selects “My Schedule” 2. Adds class by choosing day, start time, end time, and optionally course name 3. Repeats for all weekly courses 4. Clicks “Save Schedule” -> system writes data to Firebase 5. User may later edit any entry and click “Update”, or delete an entry with “Delete”
Exit Condition	Schedule stored/updated in Firebase. User’s privacy preference saved
Exception	Time conflict detected Invalid time format

- **Free Time Matching**

Name	Free Time Matching
Actors	User, System
Entry Condition	User has a weekly schedule
Event Flow	1. User navigates to “Find Matches” 2. User clicks “Find Matches” 3. System analyzes the user's schedule and identifies their free slots.

	4. System displays available dining venues -> User selects desired dining venue 5. System queries Firebase for all users who: - Have the same free time - Selected the same venue 6. System returns a list of matching students 7. If available, each student entry shows “Match” option 8. If none available -> System shows match unavailable message
Exit Condition	List of matchable students displayed or match unavailable message shown
Exception	Database errors Schedule retrieval errors



4) Design Decisions

➢ Technology Comparisons

Database: Firebase vs. MySQL

Firebase: Provides real-time data syncing and easy authentication but is less suitable for complex queries.

MySQL: A widely used relational database with high performance but requires additional setup for real-time capabilities.

Decision: Firebase was chosen due to its seamless integration with authentication and real-time updates.

Frontend Framework: React vs. Angular

React: Offers component-based architecture, a strong developer community, and efficient performance.

Angular: Provides a full-fledged MVC framework but has a steeper learning curve.

Decision: React was chosen for its flexibility and widespread industry use.

Version Control System: GitHub vs. GitLab

GitHub: Industry standard for open-source projects, great integration with CI/CD tools, large community support.

GitLab: Provides built-in CI/CD pipelines, good for self-hosting but requires more setup.

Decision: GitHub was chosen due to its ease of use, team collaboration features, and widespread adoption.

➢ **Decision Justification**

- **Frontend (HTML, CSS, React):** React provides a reusable component-based UI structure, making frontend development modular and maintainable. HTML and CSS ensure a structured and styled user interface.
- **Backend & Database (JavaScript, Firebase):** Synkly does not require a traditional backend server. Firestore provides real-time data access, secure storage, user-specific collections, and seamless integration with Firebase Authentication.
- **Version Control (GitHub):** Selected for its ease of use, issue tracking capabilities, and widespread adoption in software development workflows, and seamless integration with CI/CD tools.
- **Communication (Slack):** Enables real-time team collaboration and task updates.